

Universidad Nacional Autónoma de Honduras
Informe Técnico: Simulador de Planificación de
Procesos

Facultad de Ingeniería
Carrera: Ingeniería en Sistemas

Clase: Sistemas Operativos 1

Profesor: Ing. Elmer Padilla

Estudiantes:
Genesis Yuliana Medina Ramos
Cuenta: 20231900117

Danny Hernández
Cuenta: 20211900251

Fecha: Martes, 22 de abril de 2025

Indice

1. Introducción	3
2. Objetivos	4
3. Descripción del Sistema.....	5
3.1 Tecnologías utilizadas.....	5
3.2 Estructura	5
3.3 Estructura del Código	5
3.3.1 Estado global:.....	5
3.3.2 Funciones principales:	5
3.3.3 Visualización:	5
4. Descripción de los Algoritmos.....	6
4.1 FCFS (Primero en llegar, primero en ser atendido).....	6
4.2 SJF (El trabajo más corto primero).....	6
4.3 Round Robin (RR)	6
4.4 Prioridad.....	7
5. Cálculo de Métricas	7
6. Interfaz de Usuario.....	7
7. Diagrama de Gantt	8
8. Conclusiones.....	8
Anexos	9

1. Introducción

El presente informe describe el desarrollo de un simulador interactivo de planificación de procesos, diseñado para ilustrar el comportamiento de diversos algoritmos de planificación en un entorno de sistema operativo. Esta herramienta, desarrollada en React, permite al ingresar procesos de usuario manualmente o cargarlos desde archivos en formato CSV o JSON, ejecutar los algoritmos seleccionados y visualizar los resultados mediante un diagrama tipo Gantt.

El simulador implementa los siguientes algoritmos de planificación:

- **FCFS (First-Come, First-Served):** Ejecuta los procesos en el orden en que llegan.
- **SJF (Shortest Job First):** Ejecuta primero el proceso con la ráfaga más corta.
- **Round Robin (RR):** Asigna un quantum fijo a cada proceso, ejecutándolos de forma cíclica.
- **Prioridad:** Ejecuta primero el proceso con mayor prioridad.

Además, el simulador calcula métricas clave como el **tiempo de espera** , **tiempo de retorno** y **tiempo de respuesta** , facilitando así el análisis y comparación del rendimiento de cada algoritmo.

El simulador permite cargar datos desde archivos en formato JSON o CSV . Esto facilita la entrada masiva de procesos sin necesidad de ingresarlos manualmente.

2. Objetivos

- **Implementar** los algoritmos de planificación : FCFS , SJF , Round Robin y Prioridad .Los algoritmos de planificación: FCFS, SJF, Round Robin y Prioridad.
- **Visualizar gráficamente** el orden y la duración de los procesos mediante un diagramael orden y la duración de los procesos mediante un diagrama de Gantt.
- **Calcular métricas fundamentales** , como el tiempo de espera, tiempo de respuesta y tiempo de retorno.
- **Ofrecer una interfaz amigable** que facilita la simulación interactiva de los algoritmos .que facilita la simulación interactiva de los algoritmos.

3. Descripción del Sistema

3.1 Tecnologías utilizadas

- Frontend: ReactJS
- Gráficos: Chart.js
- Lenguaje: JavaScript

3.2 Estructura

- Componente principal: SimuladorPlanificacion
- Subcomponentes: Button, Card, CardContent
- Librería gráfica: react-chartjs-2 (para representar el diagrama de Gantt)

3.3 Estructura del Código

El código está organizado en las siguientes secciones principales:

3.3.1 Estado global:

procesos: Lista de procesos ingresados por el usuario.

resultados: Resultados de la simulación (métricas y tiempos).

algoritmo: Algoritmo seleccionado por el usuario.

quantum: Valor del quantum para el algoritmo Round Robin.

3.3.2 Funciones principales:

agregarProceso: Agregue un nuevo proceso a la lista.

simular: Ejecuta la simulación según el algoritmo seleccionado.

Funciones específicas para cada algoritmo

3.3.3 Visualización:

Tabla de procesos ingresados.

Tabla de resultados (tiempos de espera, retorno, respuesta).

Diagrama de Gantt generado con Chart.js .

4. Descripción de los Algoritmos

4.1 FCFS (Primero en llegar, primero en ser atendido)

Descripción : Ejecuta los procesos en el orden en que llegan.

Cálculos :

Inicio : $\text{Math.max}(\text{tiempoActual}, p.\text{llegada})$

Fin : $\text{inicio} + p.\text{rafaga}$

Retorno : $\text{fin} - p.\text{llegada}$

Espera : $\text{inicio} - p.\text{llegada}$

Respuesta : Igual al tiempo de espera.

4.2 SJF (El trabajo más corto primero)

Descripción : Ejecuta primero el proceso con la ráfaga más corta.

Cálculos : Similar a FCFS, pero selecciona el proceso disponible con menor ráfaga.

Inicio : $\text{Math.max}(\text{tiempoActual}, p.\text{llegada})$

Fin : $\text{inicio} + p.\text{rafaga}$

Retorno : $\text{fin} - p.\text{llegada}$

Espera : $\text{inicio} - p.\text{llegada}$

Respuesta : Igual al tiempo de espera.

4.3 Round Robin (RR)

Descripción : Asigna un quantum fijo a cada proceso. Si un proceso no termina dentro del quantum, se devuelve a la cola.

Cálculos :

Usa una cola para gestionar los procesos listos.

Actualiza el tiempo global y el tiempo restante de cada proceso.

Requiere ajustar el valor del quantum para evitar tiempos de espera largos.

4.4 Prioridad

Descripción : Ejecuta primero el proceso con mayor prioridad (menor valor numérico).

Cálculos : Similar a SJF, pero selecciona el proceso disponible con menor prioridad.

5. Cálculo de Métricas

Para cada proceso se calculan:

- Tiempo de inicio

- Tiempo de finalización

- Tiempo de Espera

- Es el tiempo que un proceso pasa esperando antes de comenzar su ejecución.

Fórmula: $\text{espera} = \text{inicio} - \text{llegada}$

- Tiempo de retorno

- Es el tiempo total que pasa un proceso en el sistema, desde su llegada hasta que termina su ejecución.

Fórmula: $\text{retorno} = \text{fin} - \text{llegada}$

-Tiempo de respuesta

- Es el tiempo que tarda un proceso en comenzar a ejecutarse después de llegar al sistema.

Fórmula: $\text{respuesta} = \text{inicio} - \text{llegada}$

6. Interfaz de Usuario

Ingreso manual de procesos, carga de archivos CSV/JSON, ejecución de simulación y visualización con gráfico tipo Gantt. La interfaz es intuitiva y guía al usuario paso a paso.

7. Diagrama de Gantt

El diagrama de Gantt muestra visualmente la ejecución de los procesos en función del tiempo. Utiliza Chart.js para generar barras horizontales que representan:

- Eje X: Tiempo.
- Eje Y: Procesos.

8. Conclusiones

- **Efectividad en la Simulación de Algoritmos de Planificación:** El simulador logra su objetivo principal al permitir la comparación visual y numérica de diferentes algoritmos de planificación de procesos (FCFS, SJF, Round Robin y Prioridad). Las métricas calculadas (tiempo de espera, retorno y respuesta) y el diagrama de Gantt proporcionan una comprensión clara del rendimiento de cada algoritmo en distintos escenarios.
- **Facilidad de Uso y Flexibilidad:** La interfaz intuitiva y las funcionalidades adicionales, como la carga de datos desde archivos JSON/CSV, hacen que el simulador sea accesible tanto para usuarios principiantes como para aquellos con conocimientos más avanzados.

9-Anexos

INTERFAZ

Simulador de Planificación de Procesos

Agregar Proceso

ID

Llegada

Ráfaga

Prioridad (opcional)

Agregar

Prioridad

Simular

Limpiar

Cargar desde archivo

Lista de Procesos

ID	Llegada	Ráfaga	Prioridad
----	---------	--------	-----------

Promedios

Tiempo de Retorno Promedio: 0

Tiempo de Espera Promedio: 0

Tiempo de Respuesta Promedio: 0

Tabla de Resultados y Diagrama de Gantt

ID	Inicio	Fin	Retorno	Espera	Respuesta
P1	0	4	4	0	0
P2	4	7	6	3	3
P3	7	12	10	5	5
P4	12	14	11	9	9

Diagrama de Gantt

ID	Inicio	Fin
P1	0	4
P2	4	7
P3	7	12
P4	12	14

Algoritmos Implementados

```
// Algoritmo FCFS
const simularFCFS = () => {
  const ordenado = [...procesos].sort((a, b) => a.llegada - b.llegada);
  let tiempoActual = 0;
  const resultado = ordenado.map((p) => {
    const inicio = Math.max(tiempoActual, p.llegada);
    const fin = inicio + p.rafaga;
    const retorno = fin - p.llegada;
    const espera = inicio - p.llegada;
    const respuesta = espera;
    tiempoActual = fin;
    return { ...p, inicio, fin, retorno, espera, respuesta };
  });
  setResultados(resultado);
};
```

```
// Algoritmo SJF
const simularSJF = () => {
  const lista = [...procesos];
  const resultado = [];
  let tiempo = 0;

  while (lista.length > 0) {
    const disponibles = lista.filter((p) => p.llegada <= tiempo);
    let siguiente;
    if (disponibles.length > 0) {
      siguiente = disponibles.reduce((a, b) => {
        if (a.rafaga === b.rafaga) {
          return a.id < b.id ? a : b;
        }
        return a.rafaga < b.rafaga ? a : b;
      });
    } else {
      siguiente = lista.reduce((a, b) => (a.llegada < b.llegada ? a : b));
      tiempo = siguiente.llegada;
    }
  }
}
```

```
// Algoritmo Round Robin
const simularRR = () => {
  const lista = procesos.filter((p) => p.rafaga > 0).map((p) => ({ ...p }));
  let tiempo = 0,
      queue = [],
      resultado = [],
      map = {};
  const done = new Set();

  while (lista.length > 0 || queue.length > 0) {
    lista
      .filter((p) => p.llegada <= tiempo && !done.has(p.id))
      .forEach((p) => {
        queue.push({ ...p });
        done.add(p.id);
      });

    if (queue.length === 0) {
      if (lista.some((p) => p.llegada > tiempo)) {
        tiempo = Math.min(
          ...lista.filter((p) => p.llegada > tiempo).map((p) => p.llegada)
        );
        continue;
      }
      break;
    }

    const actual = queue.shift();
    if (!map[actual.id]) {
      map[actual.id] = {
        ...actual,
        rafagaRestante: actual.rafaga,
      };
    }
  }
}
```

```
const simularPrioridad = () => {
  const lista = [...procesos];
  const resultado = [];
  let tiempo = 0;

  while (lista.length > 0) {
    const disponibles = lista.filter((p) => p.llegada <= tiempo);
    let siguiente;
    if (disponibles.length > 0) {
      siguiente = disponibles.reduce((a, b) => {
        if (a.prioridad === b.prioridad) {
          return a.llegada < b.llegada ? a : b;
        }
        return a.prioridad < b.prioridad ? a : b;
      });
    } else {
      siguiente = lista.reduce((a, b) => (a.llegada < b.llegada ? a : b));
      tiempo = siguiente.llegada;
    }

    const inicio = Math.max(tiempo, siguiente.llegada);
    const fin = inicio + siguiente.rafaga;
    const retorno = fin - siguiente.llegada;
    const espera = inicio - siguiente.llegada;
    const respuesta = espera;

    resultado.push({ ...siguiente, inicio, fin, retorno, espera, respuesta });
    tiempo = fin;
    lista.splice(lista.indexOf(siguiente), 1);
  }

  setResultados(resultado);
};
```